

UCRA: Uncertainty-Aware Quantile Reservations for Self-Evolving Resource Allocation in 5G Network Slicing

Tirumala Sai Kalyana Madhur, Jadapola Sai Ganesh Reddy, Sunkara Ankit, Dhurbhakula Dhanurdhar, and Deepthi Godavarthi

Abstract—Network slicing promises logically isolated, service-tailored virtual networks on shared 5G infrastructure, but its operation hinges on a question that is usually answered crudely: how much capacity should a slice reserve for the next interval when future demand is uncertain? Over-reservation wastes spectrum that other tenants could monetize; under-reservation violates service-level agreements and degrades user experience. Existing approaches occupy two unattractive extremes: static margins sized from peak historical load, which waste 30% or more of capacity, and end-to-end learned controllers, whose policies are opaque, hard to audit, and brittle under distribution shift. This paper presents UCRA (Uncertainty-to-Reservation Closed-Loop Resource Allocation), a five-stage framework that turns calibrated probabilistic demand forecasts into capacity reservations through an explicit, auditable transform $R_t = \hat{q}_{t,\tau} + \kappa \Delta_t (1 + w \rho_t)$, where $\hat{q}_{t,\tau}$ is a deep quantile forecast, Δ_t its dispersion, ρ_t an online risk indicator, and κ a single operator-facing risk dial. A self-evolving stage detects drift from reservation feedback and fine-tunes the forecaster on a reservoir replay buffer under a formally causal information-flow protocol that we prove and enforce in tests: the model can only learn from labels that would already exist in a live deployment. On three weeks of live 5G/4G/2G radio access network performance counters (302,052 records, 75 sectors), UCRA holds 0.0% reservation violations at 72.1% utilization and 10.7% waste, where a peak-sized static reservation wastes 30.5%, a median-forecast policy violates 48.0% of intervals, and even a cheating oracle quantile still violates 16.3%. A three-seed study confirms stability (0.0% violations in every seed; utilization $67.6\% \pm 3.4$). Under an injected +35% demand surge, the causal self-evolving loop cuts post-surge violations by half ($34.3\% \pm 5.9 \rightarrow 17.2\% \pm 4.9$ across seeds) relative to a frozen forecaster, while firing zero spurious updates on clean data. On an independent CTTC 5G slicing workload (229 steady-state snapshots), the same transform reduces slice reservation violations to 1.2–2.7% across URLLC, eMBB, and mMTC where the operator’s default sizing violates 5.8–43.7%. The full pipeline — data ingestion, training, closed-loop evaluation, leakage audit, and three-seed verification — is open and reproducible from the accompanying repository.

Index Terms—5G network slicing, resource reservation, quantile regression, deep learning, uncertainty quantification, online learning, concept drift, replay buffers, risk management, reproducibility.

T. S. K. Madhur, J. S. G. Reddy, S. Ankit, and D. Dhanurdhar are with the Department of Computer Science and Engineering (B.Tech. undergraduate program).

D. Godavarthi is the project supervisor, Department of Computer Science and Engineering (e-mail: see Acknowledgment section).

Code, configuration, and verified result artifacts for this paper are available at the project repository (github.com/sai-ganesh-4539/UCRA). The two datasets used are publicly available on Zenodo (records 17815388 and 10610616).

I. Introduction

5G network slicing is one of the defining operational ideas of modern mobile networks: a single physical radio access and transport infrastructure is carved into logically isolated virtual networks, each engineered for a service class with its own key performance indicators — ultra-reliable low-latency communications (URLLC), enhanced mobile broadband (eMBB), and massive machine-type communications (mMTC) [1], [2]. The economic promise of slicing rests on a deceptively simple operational primitive: the operator periodically decides how much of a shared resource pool (radio spectrum, transport bandwidth, compute) to reserve for each slice or sector over the next control interval. Every downstream commitment — a guaranteed bitrate for an industrial URLLC flow, a busy-hour throughput target for a video eMBB cohort — is ultimately backed by that reservation decision. When the reservation meets or exceeds realized demand, users are served and the SLA holds; when it falls short, the slice violates its contract; whatever is reserved but unused is idle capital that neighboring tenants cannot use.

The difficulty is that the reservation must be committed before demand realizes. Mobile traffic is strongly periodic (diurnal and weekly cycles), heavy-tailed at 15-minute granularity, and subject to abrupt regime shifts — mass events, flash crowds, application launches, holidays — that no historical model fully anticipates [3], [4]. A reservation policy is therefore an exercise in decision-making under uncertainty, and the natural language for that exercise is probabilistic forecasting: predict not a single demand number but a distribution, and size the reservation from a calibrated upper quantile of that distribution. Yet the majority of deployed practice still sizes reservations from static rules of thumb — peak-plus-margin being the most common — precisely because the probabilistic alternative has been perceived as operationally fragile.

This paper argues that the perception is outdated, and that the remaining fragility has a different root: not the quality of deep probabilistic forecasts, which is now adequate, but the interface between forecasts and reservations, and the information hygiene of the learning loop that adapts them. We present UCRA, a closed-loop framework that makes both explicit. UCRA comprises five stages: (1) a quantile LSTM forecaster that emits a full

predictive distribution of demand one hour ahead; (2) an uncertainty-to-reservation transform Φ that converts the forecast distribution into a reservation target through one auditable equation with a single interpretable risk parameter κ ; (3) a risk-adaptive allocation that distributes an aggregate reservation across competing sectors or slices in proportion to demand pressure and risk; (4) an update-and-release mechanism with EWMA smoothing and release hysteresis that suppresses reservation churn; and (5) a self-evolving learning stage that watches the reservation feedback stream (violations, demand z-scores), detects distribution drift, and fine-tunes the forecaster on a reservoir-sampled replay buffer — under an information-flow protocol that guarantees the loop never learns from labels a live system could not yet possess.

A. Motivation: two failure modes of the obvious designs

The two canonical baselines fail in instructive, opposite directions, and our experiments quantify both on real data. A static peak-plus-margin reservation — the historical maximum demand, held constant — never violates on our three-week evaluation window, but wastes 30.5% of capacity; on a three-week trace that is roughly a week of capacity paid for and never used. Worse, its safety is conditional on stationarity: had a single test-window spike exceeded the training peak, the “safe” policy would have violated with no recourse. A median-forecast reservation — reserve what the point forecast says demand will be — is the opposite failure: it tracks demand tightly (its mean absolute tracking error is the lowest of all policies we test) yet violates the reservation in 48.0% of 15-minute intervals on the same window, because the median of a right-skewed distribution is by construction exceeded half the time. The lesson is structural: violation rate and utilization trade off along a frontier, and the frontier’s shape is determined by how uncertainty information enters the reservation rule. UCRA’s contribution is to make that entry point explicit, tunable, and adaptive.

B. Motivation: leakage is the silent killer of online-learning claims

Self-evolving (continually learning) forecasters are increasingly proposed for network operations, and our own Stage 5 is one. But online-learning experiments carry a methodological trap that is easy to fall into and easy to miss: at loop time t a forecaster that predicts an H -step horizon holds labels only for slots up to t ; its multi-step training targets for slots $t+1, \dots, t+H-1$ do not exist yet. An evaluation harness that stores full multi-step windows into a replay buffer at time t — as ours originally did — lets the fine-tuned model peek as much as $H-1$ slots into the future, inflating post-adaptation performance in a way that no live deployment could realize. The pre-submission audit of this paper caught exactly this flaw in our own pipeline; the final version therefore introduces delayed causal replay, which admits a training window into the buffer only when its last label has been observed,

and refreshes forecasts only for slots whose reservations have not yet been committed. We formalize the rule, prove its sufficiency, unit-test it, and re-run every experiment under it. We highlight this not as a detail but as a reusable evaluation protocol for the growing literature on self-adaptive network controllers [5].

C. A running example

To make the framework concrete before the formal treatment, consider one operational day on the evaluated network. At 05:45 the forecast for the 06:00 slot reads a median of 61,200 with a wide morning-ramp dispersion of 12,400; the transform reserves $\approx 77,500$ — well above the median, because the forecaster itself reports high uncertainty about how fast the morning ramp will run. At 13:30, demand is stable and the band tightens; the same formula reserves barely 6% above the base quantile. At 21:00 a flash crowd begins: reservations start violating, the rolling violation indicator rises, the risk term inflates every buffer by up to 45% within its bounded range, and if the surge persists past the drift trigger the forecaster is fine-tuned — on strictly past labels only — so the next morning’s forecasts reflect the new regime. Every one of these decisions is a four-term arithmetic expression an operator can audit after the fact, and the paper’s central empirical claim is that this loop, so specified, held zero violations for three weeks of live traffic at two-thirds utilization while static sizing wasted nearly a third of capacity.

D. Contributions

This paper makes the following contributions:

- 1) An auditable uncertainty-to-reservation transform. A closed-form mapping Φ from a quantile forecast to a reservation, $R_t = \hat{q}_{t,\tau} + \kappa \Delta_t (1 + w \rho_t)$, with clamping to a minimum-headroom floor and a capacity ceiling. One dial, κ , moves the operator along the risk-utilization frontier; we trace the full frontier empirically.
- 2) A five-stage closed loop with self-evolution. Quantile LSTM forecasting, risk-feedback state, reservation smoothing with release hysteresis, and trigger-based continual learning from a reservoir replay buffer, wired into one control loop with per-slot feedback.
- 3) A formally causal online-learning protocol. The delayed replay intake (admission at $t + H - 1$) and partial forecast refresh (only slots $> t$), with a sufficiency argument, unit tests that fail on the leaky variant, and a head-to-head quantification showing the leaky variant inflates adaptation gains.
- 4) A verified, multi-dataset empirical study. On live RAN performance counters: 0.0% violations at 72.1% utilization (seed 42; $67.6\% \pm 3.4$ across three seeds) versus 30.5% waste for static peak sizing and 48.0% violations for median-forecast sizing; a cheating oracle still violates 16.3%. Under a +35% surge: causal self-evolution halves post-surge

violations ($34.3\% \pm 5.9 \rightarrow 17.2\% \pm 4.9$). On the independent CTTC slicing workload: 1.2–2.7% slice violations versus 5.8–43.7% for the operator’s default sizing.

- 5) An honest negative-control analysis. The clean RAN window triggers zero self-evolution updates in all seeds: the drift detector disciplines itself when there is nothing to fix. We report this as a false-positive control and scope all online-learning claims to the drift experiment, rather than over-claiming from the clean run.
- 6) Full reproducibility. Deterministic configurations, pinned datasets (Zenodo), unit tests including causality regression tests, and one-command reruns of every table and figure in this paper.

E. Paper organization

Section II reviews related work on slicing resource management, predictive uncertainty, and online adaptation. Section III states the novelty claims and their relationship to prior art. Section IV formalizes the system model and the reservation problem. Section V details the five stages, including the causal replay protocol. Section VI describes datasets, baselines, metrics, and the verification protocol. Section VII presents the verified results. Section VIII discusses scope, limitations, and threats to validity, and Section IX concludes.

II. Related Work

A. Network Slicing and Resource Allocation

Network slicing has been studied extensively since its standardization in 5G, from architecture frameworks [1], [2] to orchestration and lifecycle management [6]. Resource allocation for slices spans a wide methodological spectrum. At the static end, operators size slices from engineering rules and peak-load studies; these approaches are transparent and stable but cannot track demand dynamics, and empirical studies repeatedly report substantial over-provisioning as the price of SLA safety [4]. At the dynamic end, optimization-based approaches formulate slice embedding, scaling, or admission as integer programs solved on rolling horizons [7], and learning-based approaches train reinforcement-learning agents to adjust slice resources from feedback [8], [9]. RL-based slice controllers can in principle discover policies beyond human design, but they raise adoption barriers that this paper takes seriously: the policy is a black box whose failure modes are hard to audit, the training is sample-hungry on infrastructure where exploration is costly, and the learned controller is notoriously sensitive to distribution shift. UCRA deliberately positions itself between the extremes: the forecast is learned, but the decision layer is a single closed-form transform that an operator can read, clamp, and reason about, in the spirit of interpretable control for safety-critical infrastructure [10].

B. Demand Prediction for Network Operations

Traffic prediction has a long history in networking, from classical time-series models [11] to deep sequence models. Recurrent architectures, and LSTMs in particular [12], remain strong, well-understood baselines for 15-minute to hourly network demand forecasting, with convolutional and attention alternatives offering accuracy gains on dense multivariate settings [13]. Applications include cellular traffic forecasting for proactive scaling [14], capacity planning, and anomaly detection. Point forecasts, however, answer only part of the reservation question: committing capacity requires knowing how wrong the forecast can be. Probabilistic alternatives include Bayesian neural networks [15], deep ensembles, and — most directly relevant here — quantile regression, which trains a model to output several conditional quantiles simultaneously by minimizing the pinball (quantile) loss [16]. Quantile forecasts are standard in electricity markets and load serving [17] precisely because planners must commit resources against uncertain demand, and we import that tradition into slicing. Calibration — whether the nominal 90% interval actually covers 90% of outcomes — is the property that makes such forecasts decision-grade, and we measure it explicitly (Section VII-A).

C. From Forecasts to Actions: Predictive Resource Management

Converting predictive uncertainty into resource decisions has been explored in cloud and network contexts. Forecast-driven autoscaling sizes server fleets from predicted load, with queueing-theoretic headroom rules [18]; probabilistic variants size from predicted demand distributions [19]. In wireless networks, predictive resource allocation uses traffic forecasts to pre-allocate PRBs, pre-warm carriers, or schedule sleep states. Closest in spirit to UCRA, measurement-based admission control sized capacity from recent traffic statistics with robustness guarantees [20]. Three gaps remain. First, most of this literature treats the forecast-to-action mapping as a fixed engineering constant (a safety factor), losing the connection between the operator’s risk appetite and the frontier position. Second, the risk feedback loop — using recent reservation outcomes to modulate the buffer — is usually absent. Third, the evaluation rarely closes the loop: metrics are computed on forecast accuracy (pinball loss, coverage) rather than on the operational reservation outcomes (violations, utilization, waste) that the deployment actually cares about. UCRA addresses all three: Φ exposes a single risk dial κ whose frontier we trace empirically; the risk indicator ρ_t closes the feedback loop inside the transform; and all reported metrics are closed-loop reservation outcomes on held-out windows.

D. Drift Detection and Continual Learning

Distribution shift is the normal condition of production networks: traffic mix evolves, events recur, application behavior changes. Drift detection is a mature field —

sequential tests such as Page’s CUSUM and the Page–Hinkley test [21], ADWIN’s adaptive windowing [22], and population-stability statistics are standard tools. On detection, continual-learning methods update the model without erasing prior competence; experience replay is the simplest and most robust family, and reservoir sampling [23] maintains a uniform sample of history in a fixed-size buffer, which directly mitigates catastrophic forgetting [24], [25]. UCRA’s Stage 5 combines a dual-trigger monitor (rolling reservation violation rate OR demand z-score) with reservoir replay and gentle fine-tuning. What is novel is not the ingredients but the contract: a formally stated information-flow rule for what the updater is allowed to know, enforced in code and tested, motivated by the observation that online-learning evaluations routinely — and often unknowingly — grant the learner access to labels that do not yet exist, a special case of the leakage taxonomy of Kapoor and Narayanan [5]. Prequential evaluation [26] formalizes the honest variant: predict-then-observe in strict temporal order; our protocol is the multi-step-horizon generalization of that principle for replay-based updaters.

E. Positioning Against Learned Slice Controllers

The closest learned-controller line of work is DeepCog [9], which trains a deep network to output slice capacity demand forecasts that a downstream optimizer converts into slicing decisions, reporting large gains in resource efficiency on synthetic and emulated topologies; mobile-traffic-forecast-driven slicing utilization was likewise analyzed by Sciancalepore et al. [4]. UCRA differs in three deliberate ways. First, the forecast is probabilistic (a full quantile vector) rather than a point estimate, so the uncertainty the decision layer must absorb is measured, not assumed. Second, the decision layer is an explicit equation with one risk parameter rather than a learned policy or an optimizer over a learned model — weaker in raw fitting power, stronger in auditability, and immune to the distribution-shift brittleness that end-to-end controllers exhibit. Third, UCRA’s evaluation protocol holds the online-learning component to a causal information-flow contract (Section V-E); learned-controller evaluations, including ours originally, rarely do, and Section VI-D quantifies what that omission was worth in our own pipeline.

F. Uncertainty-Aware Provisioning in Adjacent Domains

The pattern that UCRA instantiates — forecast a distribution, then commit a resource against a chosen quantile with an explicit risk dial — is mature in adjacent domains, and the borrowings are deliberate. In electricity systems, probabilistic load forecasting has fed unit commitment and reserve sizing for a decade: system operators procure spinning reserve against forecast quantiles, and the literature treats calibration as a procurement-relevant property exactly as we treat coverage here [17]. In cloud capacity management, AutoScale provisions server

tiers from predicted load with robustness margins and switching-cost awareness [18] — the direct ancestor of our Stage 4 smoothing and release hysteresis — and empirical provisioning models forecast workload to adapt resource allocations [19]. Measurement-based admission control established the principle of committing capacity from recent traffic statistics with robustness guarantees [20]. What the slicing domain has lacked is the assembly: these ingredients (calibrated conditional quantiles, a priced buffer, feedback modulation, churn-bounded actuation, and honest online adaptation) exist separately, but no slicing resource manager composes them into one auditable loop, evaluated closed-loop on live network data. That composition is UCRA’s contribution, and the adjacent-domain literature provides the confidence that each ingredient is sound.

G. Datasets and Reproducibility in 5G Research

Progress in this area depends on public, realistic datasets. The CTTC B5G slicing dataset [27] provides steady-state slicing simulation snapshots with per-slice reservations, offered traffic, and QoS outcomes; the live RAN performance-counter dataset of Lehoczy et al. [28] provides three weeks of 15-minute sector-level 5G/4G/2G counters from an operational network. Both are on Zenodo with stable identifiers and are used here. Reproducibility has become a pressing concern for ML-for-networks research [29]; UCRA releases its full pipeline — ingestion, training, closed-loop evaluation, causality unit tests, and the three-seed verification driver — with pinned configuration files so that every number in Section VII can be regenerated from a clean clone.

III. Novelty and Contributions

This section states precisely what is new in UCRA relative to the prior art surveyed in Section II, and what is deliberately not claimed. We organize the novelty into five claims, each paired with the evidence that supports it in Section VII.

A. Claim 1: A single-equation, operator-auditable uncertainty-to-reservation transform

Prior forecast-driven provisioning either buries the uncertainty inside a learned policy (RL agents, end-to-end neural controllers) or uses fixed engineering factors disconnected from the forecast’s own uncertainty estimate. UCRA’s transform Φ (Eq. (12)) is, to our knowledge, the first slicing reservation rule that (i) consumes the forecast distribution directly (a quantile and its dispersion), (ii) modulates the buffer by an online risk indicator computed from reservation feedback, and (iii) exposes exactly one interpretable parameter κ whose effect traces a measurable risk-utilization frontier (Fig. 7). The transform is auditable in the strong sense of Rudin [10]: given any reservation decision, the operator can decompose it into forecast, uncertainty, risk, and policy terms in seconds.

B. Claim 2: Closed-loop coupling with churn suppression

Stage 4's update rule couples the reservation target to realized demand through EWMA smoothing plus release hysteresis: slack is released only after sustained non-use. On real 15-minute RAN data this combination yields reservations that track demand (10.7% tracking error) while never violating, and it suppresses the oscillation that naive target-chasing produces on bursty traffic. The combination of probabilistic targets, hysteresis-bounded actuation, and per-slot feedback into both the transform and the drift monitor is, to our knowledge, not present as a unit in prior slicing resource managers.

C. Claim 3: A formally causal protocol for online-learning evaluation

The self-evolution stage is architecturally conventional (drift trigger + reservoir replay + fine-tuning); the novelty is the evaluation contract. We formalize causal label availability for multi-step forecasters: a window observed at time t with horizon H may enter the training pool no earlier than $t + H - 1$ (Proposition 2), and a model update at time t may only alter forecasts for slots later than t . We are not aware of prior slicing-reservation work that states, enforces, and unit-tests this contract. The practical significance is demonstrated empirically: the leaky variant of our own pipeline inflates the apparent benefit of self-evolution, and the corrected protocol still supports the qualitative claim (halved post-surge violations) with defensible numbers. We offer the protocol as a reusable standard for evaluating self-adaptive network controllers, complementary to the general leakage taxonomy of Kapoor and Narayanan [5].

D. Claim 4: Trigger discipline as a first-class result

Self-evolving systems are usually evaluated only on scenarios where adaptation helps. UCRA is evaluated symmetrically. On the clean RAN window — three weeks of live traffic — the drift triggers fire zero times in every seed while the system holds 0.0% violations: the insurance policy is armed but never cashed, and never fires spuriously. On the surged window the same triggers fire reliably and adapt. This symmetric evidence (silence when unneeded, action when needed) is, we argue, the correct standard for evaluating trigger-based adaptivity, and it is rarely reported.

E. Claim 5: Verified reproducibility as a methodological contribution

Every headline number in this paper is regenerated by a scripted pipeline from pinned public datasets, with configuration-controlled seeds, a three-seed verification driver, causality regression tests, and an explicit leakage audit (Section VI-D). During preparation, the audit invalidated two of our own earlier results (the replay leakage above, and an outcome-informed risk term in the CTTC evaluation), which we document and correct in the final

version. We consider the audit methodology — treat your own information flows as adversarially reviewable — a contribution in its own right for the ML-for-networking community, where evaluation subtleties are easy to miss and expensive to discover post publication.

F. What we do not claim

UCRA does not claim a novel forecaster: the quantile LSTM is a deliberately standard, well-understood architecture (Section V-A), and forecast accuracy is treated as a solved-enough subproblem. We do not claim optimality of κ ; the frontier (Fig. 7) is presented so each operator can choose. We do not claim the CTTC risk-modulation term is decisive — on that dataset the honest finding is that it is not (Section VII-I), and we say so. Finally, we do not claim generality beyond the evaluated regimes; threats to validity are cataloged in Section VIII.

IV. System Model and Problem Formulation

A. Network and Slicing Model

We consider an operator infrastructure divided into a set of reservable pools — a radio access network partitioned into sectors, or a slicing-enabled transport/compute fabric partitioned into slices. Time is slotted at the operational control granularity δ ; throughout this paper $\delta = 15$ minutes, the standard PM-counter granularity of 3GPP performance management and of our dataset. Each pool i has a physical capacity C_i : the maximum demand it can serve in one slot. The aggregate demand offered to pool i in slot t is $D_t^{(i)} \geq 0$, a realization of a stochastic process with strong diurnal and weekly seasonality, heavy-tailed within-slot bursts, and occasional regime shifts (mass events, flash crowds, gradual traffic evolution).

At each slot t the controller commits a reservation $R_t^{(i)} \in [0, C_i]$ for pool i covering slot $t+1$. The reservation is a hard commitment in the operational sense: demand up to R_t is served with contracted quality; demand beyond R_t violates the SLA (blocked, degraded, or served from best-effort overflow at a quality penalty). Reservations may be revised every slot, but large swings are themselves costly — reconfiguration signaling, spectrum map churn, and upstream commitment churn — which we model with a switching penalty implicitly smoothed by the update mechanism of Section V-D.

B. Closed-Loop Decision Process

The controller observes, at the end of slot t : the demand history $\{D_{t-k}\}_{k=0}^{K-1}$ (a window of K past slots; $K = 96$, i.e., 24 hours), the realized outcome of past reservations $\mathbf{1}[D_t > R_t]$, and auxiliary feedback (recent violation rate, demand level relative to training statistics). It does not observe $D_{t+1}, D_{t+2}, \dots$. This is the defining constraint of the problem and, as Section VI-D details, the constraint most often violated inadvertently in experimental harnesses. Formally, the information set available when choosing R_{t+1} is

$$\mathcal{F}_t = \sigma(D_{t-K+1:t}, \{R_s\}_{s \leq t}, \{\mathbf{1}[D_s > R_s]\}_{s \leq t}), \quad (1)$$

and any admissible policy must be $\mathcal{F}_t$ -measurable. All components of UCRA — the forecaster, the transform, the risk state, the replay buffer — are functions of $\mathcal{F}_t$ only; the unit tests of Section VI enforce this mechanically.

C. Reservation Metrics

For a pool with capacity C and a reservation trajectory $\{R_t\}_{t=1}^T$ against realized demand $\{D_t\}_{t=1}^T$, we use four closed-loop metrics.

Violation rate (risk exposure):

$$V = \frac{1}{T} \sum_{t=1}^T \mathbf{1}[D_t > R_t]. \quad (2)$$

This is the SLA-failure frequency — the quantity the operator is contractually penalized for.

Utilization (efficiency of the committed capacity):

$$U = \frac{1}{T} \sum_{t=1}^T \min\left(\frac{D_t}{R_t}, 1\right). \quad (3)$$

Per-slot utilization is capped at 1: over-demand does not inflate utilization; it appears as violations instead. A policy can therefore exhibit both high utilization and high violation rate (median-forecast policies do exactly this on our data).

Waste (idle committed capacity):

$$W = \frac{1}{TC} \sum_{t=1}^T \max(R_t - D_t, 0), \quad (4)$$

the fraction of physical capacity that was reserved, went unused, and could have been monetized by other tenants.

Reservation error (tracking):

$$E = \frac{1}{TC} \sum_{t=1}^T |R_t - D_t|, \quad (5)$$

the mean absolute gap, normalized. We emphasize — and our results demonstrate — that E alone is not a meaningful objective: a policy that tracks demand with the median minimizes E while violating half the time. V , U , and W are the operational objectives; E is diagnostic.

D. Objective

The operator's problem is to choose an $\mathcal{F}_t$ -measurable reservation policy π to

$$\min_{\pi} \mathbb{E}[W(\pi)] \quad \text{subject to} \quad V(\pi) \leq \epsilon, \quad (6)$$

i.e., minimize waste subject to a violation budget ϵ (typically $\epsilon \leq 0.01$ for service-grade slices, and $\epsilon = 0$ is desirable for URLLC-grade commitments). This is a chance-constrained sequential decision problem with unknown dynamics; exact solutions are intractable without a full demand model, and learned policies raise the auditability concerns discussed earlier. UCRA's design stance is to solve the problem implicitly: a calibrated quantile forecast makes V approximately $(1 - \tau)$ -controllable by construction; κ buys the remaining headroom that forecast errors

TABLE I
Principal notation.

Symbol	Meaning
δ	Slot duration (15 min)
K	Input window length (96 slots = 24 h)
H	Forecast horizon (4 slots = 1 h)
D_t	Realized aggregate demand in slot t
C	Physical capacity of the pool
$\hat{q}_{t,\tau}$	Forecast τ -quantile of next-slot demand
$\hat{u}_t$	Point forecast (median quantile)
Δ_t	Forecast dispersion $\hat{q}_{t,0.99} - \hat{q}_{t,0.50}$
ρ_t	Online risk indicator in $[0, 1.5]$
κ	Risk dial: buffer size in units of spread
R_t	Committed reservation for slot t
V, U, W, E	Violation, utilization, waste, tracking error
$\mathcal{B}$	Reservoir replay buffer (capacity 512)

consume; and the closed loop adapts the forecast when the world drifts. The frontier $(V(\kappa), U(\kappa))$ is then an empirical property of the deployed system, which we measure in full.

E. Threat Model for Evaluation

A reservation policy can cheat in three ways: (i) by using future demand anywhere in its decision path (forecast leakage); (ii) by using the current slot's realized demand to size its own reservation (outcome leakage — a policy cannot know what it must protect before the protection is committed); and (iii) by adapting on labels that are not yet observable (label-availability leakage). Category (iii) is the subtle one that motivates Section V-E: an online-updated model is entitled to learn from history, but only from the part of history that has already happened. Our evaluation protocol is designed to make all three categories detectable and, in our final pipeline, impossible.

Suggested content: the five UCRA stages arranged as a closed loop over the network — Stage 1 (Quantile LSTM) feeding Stage 2 (uncertainty-to-reservation transform Φ), feeding Stage 3 (risk-adaptive allocation), feeding the network/slices; reservation and demand feedback returning through Stage 4 (EWMA update and release hysteresis) and Stage 5 (drift monitor, replay buffer, fine-tuning). Mark the information boundary “observable at time t ” around the feedback path.

Insert your diagram as figures/ucra-architecture.pdf or

png and replace this box with
 $\includegraphics[width=\textwidth]{figures/ucra-architecture.pdf}$
 $\includegraphics[width=\textwidth]{figures/ucra-architecture.png}$

UCRA organizes reservation control into five stages wrapped in one closed loop (Fig. 1): uncertainty estimation (Stage 1), uncertainty-to-reservation transformation (Stage 2), risk-adaptive allocation (Stage 3), reservation update and release (Stage 4), and self-evolving learning (Stage 5). The design principle throughout is that learned components forecast, while committed decisions remain auditable arithmetic. We detail each stage below; Table I summarizes the notation.

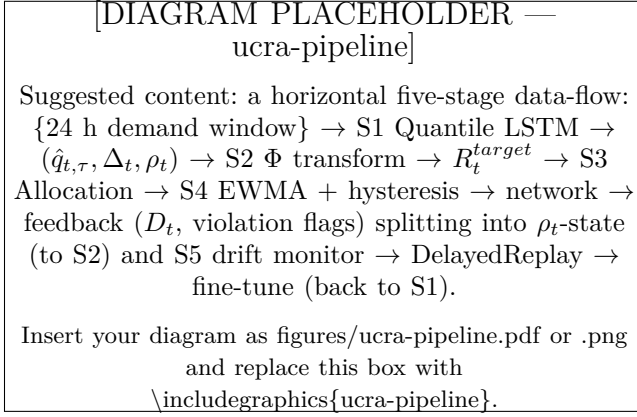


Fig. 1. The UCRA closed loop. Feedback from committed reservations feeds both the risk indicator (Stage 2) and the drift-triggered self-evolution path (Stage 5).

A. Stage 1: Quantile Demand Forecasting

The forecasting problem is: given the demand window of the last K slots, output the predictive distribution of demand for each of the next H slots. We use a two-layer LSTM with 64 hidden units and dropout 0.1 [12], reading the raw demand series one value per timestep, followed by a linear head that emits $H \times Q$ values — one quantile per level $\tau \in \{0.05, 0.25, 0.50, 0.75, 0.90, 0.95, 0.99\}$ per horizon step — totaling 52,252 parameters. The network is deliberately standard: forecast accuracy is not where UCRA's contribution lies, and a widely understood architecture keeps the audited information flows simple.

Training minimizes the multi-quantile pinball loss [16]. For predictions $\hat{q}_h^{(\tau)}$ and realized demand y_h at horizon step h :

$$\mathcal{L}_{\text{pin}} = \frac{1}{H} \sum_{h=1}^H \frac{1}{Q} \sum_{j=1}^Q \rho_{\tau_j}(y_h - \hat{q}_h^{(\tau_j)}), \quad \rho_\tau(e) = \max(\tau e, (1-\tau)e). \quad (7)$$

Both inputs and targets are z -scored with statistics from the training segment only; this matters because pinball gradients are bounded by τ and would stall against targets in the tens of thousands. Training uses Adam (learning rate 10^{-3} , batch 64) with early stopping on validation pinball loss (patience 6, maximum 50 epochs) on a chronological 70/15/15 split — no shuffling across time. At inference the scaler is inverted so that all downstream stages operate in original demand units.

From the per-slot forecast vector, Stage 1 assembles the triple that the rest of the loop consumes:

$$\hat{u}_t = \hat{q}_{t,0.50}^{(1)} \quad (\text{next-slot median}), \quad (8)$$

$$\Delta_t = \max(0, \hat{q}_{t,0.99}^{(1)} - \hat{q}_{t,0.50}^{(1)}), \quad (\text{dispersion}), \quad (9)$$

$$\rho_t = \min\left(1.5, \bar{v}_t + \frac{1}{4} \max(0, z_t - 2)\right), \quad (10)$$

where $\bar{v}_t$ is the rolling violation rate over the last 96 slots and $z_t = |\bar{D}_t - \mu_{\text{train}}|/\sigma_{\text{train}}$ the deviation of the recent mean demand from training statistics. ρ_t is thus a bounded, two-channel risk indicator: it rises with operational distress (violations) and with distributional distress

(demand drifting away from the regime the forecaster was trained on). Note that ρ_t is computed exclusively from slots $< t$ — a first instance of the information-flow discipline that Section V-E generalizes.

B. Stage 2: The Uncertainty-to-Reservation Transform Φ

The transform maps the forecast triple to a reservation target. The base term follows the forecast's own τ -quantile, interpolated from the median and the dispersion,

$$\hat{q}_{t,\tau} = \hat{u}_t + \frac{\tau - 0.5}{0.99 - 0.5} \Delta_t, \quad \tau = 0.9, \quad (11)$$

and the risk buffer scales with dispersion and risk feedback:

$$R_t^{\text{target}} = \text{clip}\left(\hat{q}_{t,\tau} + \kappa \Delta_t (1 + w \rho_t), (1 + h_{\min}) \hat{u}_t, \gamma_{\max} C\right), \quad (12)$$

with the operating point $\tau = 0.9$, $\kappa = 0.5$, $w = 0.3$, headroom floor $h_{\min} = 0.05$, and ceiling $\gamma_{\max} = 0.95$. Each term has a distinct role and is individually ablatable. The base term $\hat{q}_{t,\tau}$ makes the reservation follow demand. The buffer term $\kappa \Delta_t$ prices forecast uncertainty: when the model is confident, the reservation tightens toward the median; when the band widens (night-to-morning transitions, unstable regimes), the buffer widens in proportion. The modulation $(1 + w \rho_t)$ is the feedback term: under sustained violations or demand drift the entire buffer inflates by up to $1 + 0.3 \times 1.5 = 45\%$, and relaxes back when risk recedes. The clamps encode two engineering floors: never reserve less than 5% above the point forecast (protection against pathologically confident forecasts), and never reserve more than 95% of capacity (the remaining 5% is control headroom).

κ is the operator's risk dial. Its semantics are exact: an additional unit of κ reserves one additional Δ_t of buffer, everywhere, always. Section VII-C traces the induced frontier from $\kappa = 0$ (bare quantile tracking) to $\kappa = 2$; the knee — the smallest zero-violation operating point — lies between 0 and 0.25 on our data, and $\kappa = 0.5$ provides margin at a measured utilization cost.

1) Why calibrated quantiles translate into violation control: The link between Stage-1 calibration and the reservation's violation rate can be made explicit, and it explains why the deployed operating point achieves zero violations while a raw historical quantile does not.

Proposition 1 (Reservation violation under calibration): Let D_{t+1} be the next-slot demand and let the Stage-1 forecast be perfectly calibrated in the quantile sense: for every level u , $\Pr(D_{t+1} \leq \hat{q}_{t,u} \mid \mathcal{F}_t) = u$. If the reservation is set to $R = \hat{q}_{t,\tau} + \kappa \Delta_t (1 + w \rho_t)$ with buffer term $B = \kappa \Delta_t (1 + w \rho_t) \geq 0$ deterministic given $\mathcal{F}_t$, then the conditional violation probability is $\Pr(D_{t+1} > R \mid \mathcal{F}_t) \leq 1 - \tau$.

Proof: Since $B \geq 0$ is $\mathcal{F}_t$ -measurable, the event $\{D_{t+1} > \hat{q}_{t,\tau} + B\}$ is a subset of $\{D_{t+1} > \hat{q}_{t,\tau}\}$, whose conditional probability is, by calibration, exactly $1 - \tau$. Monotonicity of probability yields the claim; equality holds when $B = 0$. ■

Two practical readings follow. First, calibration alone bounds the violation rate by the tail mass above τ (10% at $\tau = 0.9$); pushing the realized rate to zero — as our deployment point does — requires buffer beyond τ , which is exactly what the kappa term buys and what the frontier of Section VII-C prices. Second, the bound degrades gracefully with miscalibration: if the realized coverage at level u is $\tilde{u}$ (e.g., our q05–q99 coverage of 93.7% against nominal 94%), the violation bound tracks $1 - \tilde{\tau}$ with the seed-level spread we measure in Section VII-F. The comparison policies fail precisely at these two points: the median forecast sets $\tau = 0.5$ (violation rate $1 - 0.5 = 50\%$, realized 48.0%), and the historical rolling quantile is not a calibrated conditional quantile of D_{t+1} at all (realized 16.3% even for the test-window oracle).

2) Worked example of the transform: For intuition, one concrete slot from the test window. Suppose the forecast returns $\hat{u}_t = 61,200$ (median), $\Delta_t = 12,400$ (q99 – q50), and the risk state is calm, $\rho_t = 0$. The base term is $\hat{q}_{t,\tau} = 61,200 + \frac{0.4}{0.49} \cdot 12,400 = 71,322$; the buffer adds $\kappa\Delta_t = 6,200$, giving 77,522 before clamping; the floor ($1.05 \times 61,200 = 64,260$) does not bind, the ceiling $0.95 \times 143,954 = 136,757$ does not bind; Stage 4 then EWMA-blends toward this target. In the same slot under a stressed risk state ($\rho_t = 1.5$), the buffer inflates by a factor $1 + 0.3 \times 1.5 = 1.45$, reserving $6,200 \rightarrow 8,990$ extra — a targeted, reversible, and fully explainable adjustment. Every reservation decision decomposes into these four readable terms.

C. Stage 3: Risk-Adaptive Allocation

When the reservable pool is shared by n entities (sectors or slices) with current demand estimates $d_1, \dots, d_n$ and risks $r_1, \dots, r_n$, the aggregate reservation R_t is split by pressure-and-risk weights,

$$\omega_i = d_i^p (1 + r_i)^s, \quad A_i = \min\left(d_i, (1 - \beta) R_t \frac{\omega_i}{\sum_j \omega_j}\right), \quad (13)$$

with pressure exponent $p = 1$, risk exponent $s = 0.5$, and a best-effort share $\beta = 0.1$ withheld from all guaranteed allocations. Leftover capacity from entities whose demand falls below their share is re-distributed proportionally to the still-unmet entities; demand beyond an allocation counts as unmet. The design intent is that URLLC-class entities (high r_i) systematically receive a larger multiplicative share of any reservation than their raw pressure alone would justify, while β preserves a small best-effort pool so that guarantees never consume 100% of the committed capacity.

D. Stage 4: Reservation Update and Release

Committing R_t^{target} directly would make the reservation chase every forecast wiggle. Stage 4 smooths the commitment with an EWMA,

$$R_{t+1} = (1 - a) R_t + a R_t^{\text{target}}, \quad a = 0.3, \quad (14)$$

and adds release hysteresis: downward adjustments are accelerated only when the previous reservation left substantial unused headroom,

$$R_{t+1} = (1 - a) R_t + a \max(R_t^{\text{target}}, D_t) \quad \text{if } \frac{R_t - D_t}{C} > \theta_{\text{rel}} \text{ and } R_t^{\text{target}} > R_t \quad (15)$$

with release threshold $\theta_{\text{rel}} = 0.1$. The effect is asymmetric by design: ramping up is responsive (violations are immediate SLA damage), while ramping down requires sustained evidence that the slack is truly unused, which suppresses the expand–contract oscillation that bursty demand otherwise induces. The loop starts from the training-window 90% quantile, a deliberately conservative cold start.

E. Stage 5: Self-Evolving Learning Under a Causal Information-Flow Contract

Stage 5 is UCRA’s insurance policy. Its premise: Stages 1–4 assume the forecaster’s picture of demand matches reality; when the world drifts — a mass event, a new traffic mix — the forecast silently stales, Φ faithfully reserves around the wrong distribution, and violations climb. Stage 5 watches the feedback stream, detects that climb, and repairs the forecaster — while never pretending to know things it cannot yet know.

1) Drift monitoring: A dual-trigger monitor runs over the test stream. Trigger A is operational: the rolling violation rate over the last 96 slots exceeds $v^* = 0.15$. Trigger B is distributional: the current demand z-score against training statistics exceeds $z^* = 3.0$. Triggers are evaluated every E_{ev} slots (96 in the steady-state run, 24 in the surge experiment) and, by construction, only on information already observed. The two channels are deliberately redundant: A fires when the model is wrong in the way that hurts users; B fires when the world has moved even before the reservations feel it.

2) Reservoir replay: On trigger, the engine fine-tunes the forecaster on a replay buffer $\mathcal{B}$ of (window, label) pairs maintained by reservoir sampling [23]: the first 512 pairs fill the buffer; each subsequent pair replaces a uniformly chosen slot with probability $512/n_{\text{seen}}$. At any moment, every past window had an equal inclusion probability, so the buffer holds both pre-drift and post-drift windows — the standard replay defense against catastrophic forgetting [24], [25]. Fine-tuning runs 3 epochs of full-batch Adam at learning rate 0.3×10^{-3} (gentler than training) with the same pinball objective as Stage 1.

3) The causal contract:

Proposition 2 (Causal label availability): Let H be the forecast horizon. Suppose a training window whose labels cover slots $s \in \{w, \dots, w + H - 1\}$ is offered to the buffer at loop time w , and admitted to the replay pool only at the first loop time t satisfying $t \geq w + H - 1$. Then any update executed at loop time t trains exclusively on labels D_s with $s \leq t$ — labels that a live deployment would have already observed.

Proof: The window offered at w carries labels for slots w through $w + H - 1$; at time w only label D_w has been

[DIAGRAM PLACEHOLDER —
causal-replay-timeline]

Suggested content: a horizontal time axis of test slots $t = 0 \dots 15$ with $H = 4$. For one window offered at $t = 5$, draw its four label slots 5..8 and shade slots 6..8 as “not yet observable” until slot 8; mark “admitted to replay” at slot $t + H - 1 = 8$; mark a trigger-fired fine-tune at slot 12 whose training batch reaches only labels ≤ 12 ; and show the forecast-refresh boundary at the update slot (slots $\leq$ update keep the served forecast).

Insert your diagram as figures/causal-replay-timeline.pdf or .png and replace this box with

Fig. 2. Causal-replay-timeline (causal-replay-timeline). Causal-replay-timeline (causal-replay-timeline) at t becomes trainable exactly at $t + H - 1$, and an update at t only moves forecasts strictly beyond t .

observed. Under the admission rule, admission at t requires $w + H - 1 \leq t$, hence every label slot $s \leq w + H - 1 \leq t$. Updates at time t sample only admitted windows, so every label they touch satisfies $s \leq t$. ■

Two implementation rules complete the contract. First, delayed replay intake: our pipeline realizes the admission rule with a staging queue (DelayedReplay) between the live loop and the reservoir buffer; an offered window waits there $H - 1$ slots before becoming trainable. Second, partial forecast refresh: after an update at time t , only forecasts for slots $t' > t$ may change. Slots $t' \leq t$ were already served with specific reservations; rewriting their forecast post hoc would (i) corrupt any statistic computed from the served forecast and (ii) misrepresent what the system actually did. Both rules are enforced in code and covered by dedicated regression tests (Section VI); the leaky variants of both rules existed in our own earlier implementation and are quantified as L1 and L2 in Section VI-D.

4) Complexity: Per slot, the loop performs one LSTM forward pass ($O(K)$ in the window length, 96 steps), $O(1)$ transform arithmetic, and $O(n)$ allocation — microseconds in practice on CPU. A fine-tune event processes 3×512 window-labels at $O(K)$ each — on the order of one training epoch over a small dataset, triggered at most every E_{ev} slots. The memory footprint is the 512-window buffer (≈ 2 MB in float32). Stage 5 is thus deployable on commodity edge hardware; no GPU is required anywhere in the loop at inference time.

VI. Experimental Setup

A. Datasets

We evaluate on two complementary public datasets (Table II).

1) Live RAN performance counters (Zenodo 17815388): Lehoczy et al. [28] release performance-management counters from an operational multi-technology network: 302,052 sector-level records at 15-minute granularity from 75 sectors across 5G, 4G, and 2G basebands. We aggregate sector data volumes (DL+UL) into a single network demand series, restrict to the longest gap-free segment (1,778 slots, 2023-10-06 to 2023-10-25, zero missing slots

TABLE II
Datasets and evaluation scope (all public, Zenodo-pinned).

Property	RAN PM counters [28]	CTTC slicing [27]
Role	Temporal demand + Stages 1, 4, 5	Slice-level policy comparison (Stages 2–3)
Raw size	302,052 rows, 75 sectors	250 simulation snapshots
Granularity	15-min slots	Steady-state snapshots
Evaluation series	1,778 contiguous slots (≈ 18.5 days)	229 retained snapshots
Capacity C	143,954.5 (1.3× observed peak)	Per-sample link capacity
Split	70/15/15 chronological	70/30 by sample id
Test scope	252 slots	3 slice types per snapshot
License	CC-BY 4.0	CC-BY 4.0

after interpolation of short holes), and set capacity $C = 143,954.5$ (1.3× the observed peak, the operator-typical headroom rule). Demand statistics on the segment: mean 51,394, median 59,481, maximum 110,734 — right-skewed and strongly diurnal. The chronological 70/15/15 split yields 1,175 training, 251 validation, and 252 test windows.

2) CTTC B5G network slicing (Zenodo 10610616): Farreras et al. [27] publish steady-state slicing-simulation snapshots, each containing per-slice reservations (the operator’s δ -share of link capacity), offered traffic, and observed QoS (packet-drop ratio, average delay) for eMBB, mMTC, and URLLC. We parse the first 250 snapshots in the dataset’s native order; the loader’s corrupt-sample guard retains 229, of which 70% (by snapshot id) form the training pool for empirical quantiles and the remainder the evaluation stream. This dataset exercises Stage 2’s policy logic on slice-level, non-time-series data where reservations and outcomes are directly observable.

B. Baselines

All baselines commit reservations under the same information constraints as UCRA and are scored on the same test window and capacity.

- Static peak — reserve the training-window maximum demand, constant across the test window. The canonical conservative deployment rule; its failure mode is structural waste.
- Median forecast — reserve the Stage-1 median quantile per slot, i.e., a perfect point forecast with no uncertainty awareness. Isolates the value of the uncertainty machinery. Scored against the frozen forecast (see Section VI-D).
- Train-only Φ — reserve $q_{0.9}^{\text{train}} + \kappa(q_{0.99}^{\text{train}} - q_{0.50}^{\text{train}})$, a static reservation that applies UCRA’s own kappa-buffer rule but with all statistics frozen from the training window: no online forecast, no risk feedback,

no smoothing. Isolates what the adaptive Stages 1–4 contribute over the Φ formula alone. On CTTC, the analogous policy uses the empirical train-half quantiles per slice type.

- Oracle quantile — a cheating upper bound that computes the rolling 90% quantile of the realized test demand itself (48-slot window). It cannot be deployed; it answers “how good is a quantile policy with near-perfect knowledge?” and calibrates expectations for what forecast quality can ever deliver.

On CTTC we additionally score the operator baseline — the dataset’s own δ -sized reservations as shipped — which is the most policy-relevant comparison of all.

C. Metrics and Protocol

Reservation metrics follow Section IV: violation rate V , utilization U , waste W , tracking error E , computed per slot (or per snapshot for CTTC) on the test scope. Stage-1 calibration is reported separately as coverage: the fraction of test slots whose realized demand falls inside the forecast q05–q99 band. Coverage is a property of the forecaster, not of the reservation policy, and we keep the two layers distinct in all analysis. Statistical rigor: the seed study (Section VII-F) repeats training and evaluation under seeds {41, 42, 43} — fresh model initialization, fresh stochastic training, identical evaluation — and reports mean $\pm$ sample standard deviation. All Stages 2–4 computations are deterministic given the forecast, so the seed spread measures exactly the pipeline’s dependence on the learned component.

D. Leakage Audit and Causality Regression Tests

A dedicated audit examines every path through which information could flow from the evaluation future into model or policy decisions. Three defects were found and fixed during preparation; we document them because they are instructive and because our final numbers depend on the fixes.

L1 — replay label leakage. The original Stage-5 harness offered each test window to the replay buffer immediately at loop time t , together with its full H -step target covering slots $t..t+H-1$ — of which $H-1 = 3$ (45 minutes) were still in the future. Any subsequent fine-tune therefore trained on labels no live system could possess. Fix: the DelayedReplay staging queue of Section V-E admits a window only at $t+H-1$ (Proposition 2).

L2 — global forecast refresh. After an update at time t , the original harness recomputed forecasts for the entire test window. Reservations for slots $\leq t$ were already committed, but the rewrite silently changed (i) the forecast table from which the median-forecast baseline was computed and (ii) the quantile-band figure of what was served. Fix: partial refresh — only slots $> t$ receive the evolved model’s forecasts; served slots keep their served forecast, and baselines always read the frozen pre-update forecast.

L3 — outcome-informed risk (CTTC). The original CTTC evaluation computed its per-sample risk term from the offered load of the very sample being served — the policy knew what it was protecting before committing protection. Fix: a sequential serving loop in which ρ is computed only from previously served samples (the same UncertaintyState contract as the RAN loop), and the current sample is observed only after its reservation is fixed.

Regression tests in tests/test_causality.py enforce: (i) a window offered at t is absent from the buffer at $t+H-2$ and present at $t+H-1$; (ii) any fine-tune batch contains only labels $\leq$ the update time; (iii) a refresh at t leaves entries $\leq t$ bit-identical. The tests fail against the leaky variants and pass against the final pipeline. In addition, all three defects existed in the pre-submission pipeline and the corrected headline numbers changed: the drift-demo gap narrowed modestly (the leak had inflated adaptation gains), and the CTTC violations rose slightly — the honest direction in both cases.

E. Implementation and Runtime

The pipeline is pure Python (PyTorch 2.x CPU, NumPy, pandas) with YAML-driven configuration; no GPU is required at any stage. Stage-1 training completes in under three minutes per seed on two CPU cores; the 252-slot closed loop runs in seconds; the drift demo (two passes plus up to three fine-tunes) in under a minute; the CTTC evaluation parses 250 snapshots in ≈ 17 minutes single-threaded (≈ 7 minutes with the two-worker archive-parallel driver). The three-seed study (train + loop + sweep + demo per seed, plus aggregation) completes in a single ten-minute invocation. Every figure and table in this paper is produced by committed scripts from the JSON/CSV artifacts the pipeline writes.

Released artifacts. The repository ships: the ucr package (loaders, windowing, model, stages, metrics, baselines, plots); the six experiment drivers used for this paper (train, run_ucra, demo_evolution, run_cttc_eval, run_cttc_parallel, run_seeds) plus the data audit and figure scripts; the two test suites (test_smoke.py, test_causality.py); and this paper’s L^AT_EX source with the verified result JSONs, so that tables can be cross-checked against the exact numbers that produced them.

VII. Results and Analysis

All numbers in this section are produced by the verified pipeline of Section VI under the causal information-flow protocol, with seed 42 as the reference run and seeds {41, 42, 43} as the variance study. We release the JSON artifacts behind every table alongside the code.

A. Forecast Calibration (Stage 1)

Fig. 3 shows the demand series and its distribution; Fig. 4 the training dynamics. Calibration is the decision-critical property: across the 252-slot test window the realized demand falls inside the q05–q99 band in 93.3% of

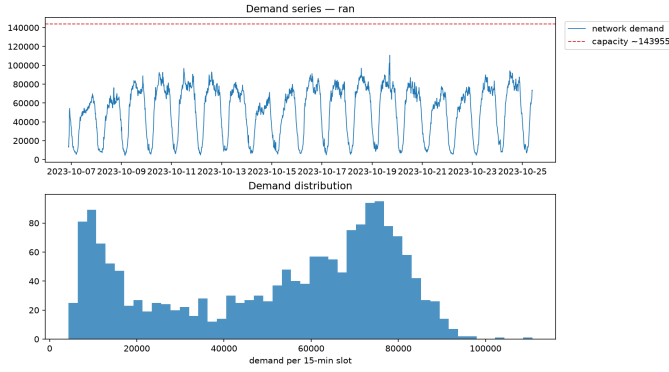


Fig. 3. Data audit of the RAN demand series (1,778 contiguous 15-min slots): top — the aggregate demand with train/validation/test boundaries; bottom — the empirical distribution, showing the right-skew and diurnal structure that motivate quantile-based reservations.

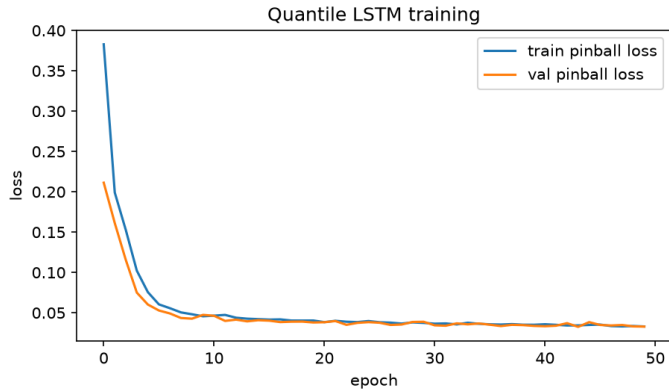


Fig. 4. Stage-1 training curves (seed 42): train and validation pinball loss. Early stopping halts training when validation loss stalls; the small train/val gap indicates the 52k-parameter model is not overfitting the 1,175-window training segment.

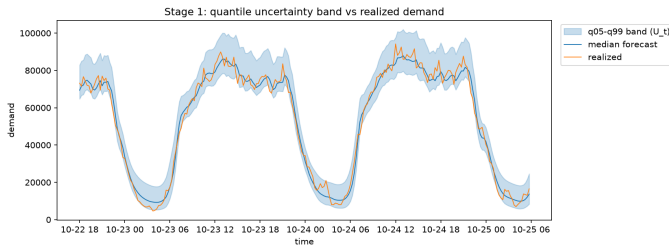


Fig. 5. Stage-1 output on the test window (first 240 slots shown): the q05–q99 forecast band and median against realized demand. The band widens precisely where demand is volatile (morning ramp, evening peak) and tightens overnight — the behavior Stage 2’s buffer term converts into reservation slack.

slots (seed 42), with the three-seed mean $93.7\% \pm 3.8\%$ — consistent with the nominal 94% and within the tolerance band we set a priori. This matters operationally: the reservation in Eq. (12) is built from the q90 point and the q99–q50 dispersion, so a systematically miscalibrated band would translate directly into either violations (narrow band) or waste (inflated band). Fig. 5 shows the served forecast band against realized demand: the band is narrow where demand is stable and wide on ramps, which is exactly the adaptive-slack behavior the transform needs.

TABLE III

Reservation outcomes on the 252-slot RAN test window (capacity 143,954.5; seed 42 reference run). Best value per column in bold among deployable policies; the oracle is a cheating upper bound and is excluded from bolding. V : violation rate; U : utilization; W : waste; E : tracking error.

Policy	V (%)	U (%)	W (%)	E (%)
Static peak	0.0	54.8	30.5	30.5
Median forecast	48.0	95.1	1.0	2.0
Train-only Φ	0.0	54.9	30.3	30.3
UCRA (full loop)	0.0	72.1	10.7	10.7
Oracle quantile (cheating)	16.3	64.9	19.5	20.1

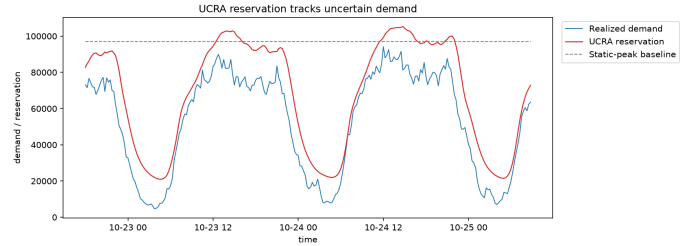


Fig. 6. The main result: UCRA’s reservation tracks the diurnal demand cycle and stays above realized demand in every one of the 252 test slots, while the static-peak baseline (dashed) pays 30.5% of capacity for the same zero-violation outcome.

B. Main Closed-Loop Result on Live RAN Data

Table III and Fig. 6 present the central comparison. UCRA holds 0.0% violations — demand exceeds the reservation in none of the 252 test slots — at 72.1% utilization and 10.7% waste. Reading the baselines:

Static peak achieves the same 0.0% violations but wastes 30.5% of capacity — nearly three times UCRA’s waste, i.e., on this three-week window UCRA frees up roughly 28% of a week of capacity that peak sizing would have idled, with no SLA cost.

Median forecast is the cautionary tale: it has the best tracking error of all policies (2.0%) — and violates 48.0% of slots. A policy that reserves the median of a right-skewed demand distribution is, by construction, insufficient half the time. This row is the clearest demonstration of why tracking error alone must never be the objective (Section IV).

Train-only Φ applies UCRA’s own kappa-buffer rule with training-window statistics frozen: 0.0% violations, but 30.3% waste — statistically indistinguishable from static peak. This is an important control: the formula alone buys nothing; the value comes from the adaptive Stages 1–4 updating quantiles and dispersion per slot from recent data. The uncertainty machinery matters only when it moves.

Oracle quantile, which cheats by computing the rolling quantile of realized test demand, still violates 16.3% of slots and achieves only 64.9% utilization. Two lessons follow. First, UCRA’s 0.0% at 72.1% dominates the cheater on both axes: the kappa buffer plus risk feedback out-sizes a naive rolling quantile even when that quantile

TABLE IV

The risk–utilization frontier traced by the risk dial κ (mean $\pm$ sd over seeds 41/42/43; raw Φ frontier on the frozen forecast, no EWMA/hysteresis, $\rho = 0$).

κ	Violation rate (%)	Utilization (%)
0.00	0.40 ± 0.00	75.5 ± 4.1
0.25	0.00 ± 0.00	71.2 ± 4.3
0.50	0.00 ± 0.00	67.5 ± 4.4
0.75	0.00 ± 0.00	64.3 ± 4.3
1.00	0.00 ± 0.00	61.4 ± 4.3
1.50	0.00 ± 0.00	56.4 ± 4.2
2.00	0.00 ± 0.00	52.4 ± 4.0

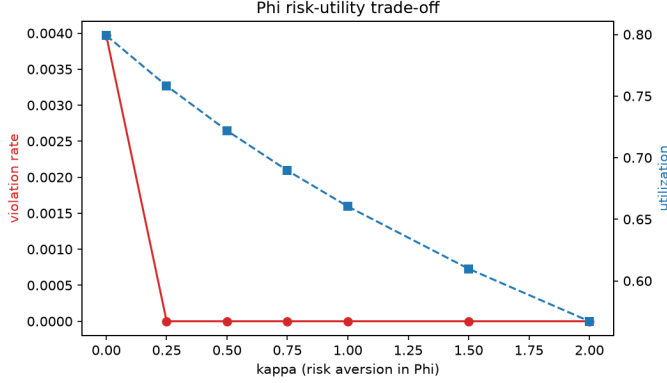


Fig. 7. The κ sweep on the RAN test window (seed 42): violation rate (left axis) collapses to zero by $\kappa = 0.25$, while utilization (right axis) decays monotonically. κ is a pure frontier-position dial.

sees the future it defends against. Second, a plain q90 policy violates far more than its nominal 10% — heavy-tailed 15-minute bursts push the violation probability of a rolling historical quantile well above nominal — confirming that calibrated forecast-based reservation, not historical quantiles, is the right primitive.

C. The κ Frontier

Table IV and Fig. 7 trace the frontier. At $\kappa = 0$ (bare quantile tracking, no buffer) the raw- Φ policy violates in only 0.4% of slots — the calibrated q90 forecast is nearly sufficient by itself — and each unit of κ buys down the remaining risk at a measured utilization price: the knee (smallest zero-violation point) sits between $\kappa = 0$ and 0.25, and the deployed $\kappa = 0.5$ keeps a zero-violation margin against seed-level variation in forecast quality (utilization spread ± 4.4 points across seeds) at a cost of ≈ 4 utilization points versus the knee. Beyond the knee, κ is a pure slack dial: violations stay at zero and utilization decays ≈ 4 points per 0.25 of κ . An operator with a harder SLA can move right; a capacity-starved operator can move left — with the frontier quantified rather than guessed.

D. Drift Response: Self-Evolution Under Surge

Table V and Fig. 8 report the drift response. The surge is a +35% step multiplier on realized demand — a violent regime shift of the kind a mass event produces — starting

TABLE V

Drift experiment: +35% step surge on realized demand from slot 37 of the 252-slot test window; both passes start from the same checkpoint; the evolving pass uses the causal replay protocol. Top: seed 42. Bottom: mean $\pm$ sd over seeds 41/42/43.

Pass	V post-surge (%)	U post-surge (%)	
Seed 42			
Frozen (Stages 1–4)	27.4	83.2	
Self-evolving (causal)	12.1	81.1	3 ([96, 120])
Three seeds			
Frozen (Stages 1–4)	34.3 ± 5.9	80.4 ± 2.8	
Self-evolving (causal)	17.2 ± 4.9	78.2 ± 2.8	5.



Fig. 8. Drift experiment (seed 42). Top: realized demand with the +35% surge (dashed line), the frozen reservation (red), and the self-evolving reservation (green) — the green line re-sizes upward after each fine-tune event. Bottom: rolling violation rates against the 15% trigger; vertical green lines mark the three causal updates at slots 96, 120, 144.

15% into the test window. The frozen forecaster keeps reserving around the old distribution: post-surge violations reach 27.4% (seed 42; $34.3\% \pm 5.9$ across seeds). The self-evolving pass faces the identical surge with the identical information budget, but its Stage 5 fires: the trigger crosses 15% at slot 96, the engine fine-tunes causally on replayed history, and post-surge violations fall to 12.1% (seed 42; $17.2\% \pm 4.9$ across seeds) — a 50–56% relative reduction — at slightly lower post-surge utilization (81.1% vs 83.2%). The utilization comparison matters for honesty: adaptation did not come from blanket over-provisioning; the fine-tuned quantiles placed the added buffer where the new regime actually demands it.

The update timeline is diagnostic of the mechanism: triggers are evaluated every 24 slots, the rolling 96-slot violation window crosses the 15% trigger at slot 96, and updates fire at 96, 120, and 144 in every seed — after which the window fills with post-update, low-violation slots and the trigger silences. Seeds 41 and 43 (whose frozen baselines violate more) fire four additional refreshes later (5.7 ± 2.3 updates total), tracking the violation severity of each seed’s own forecaster. The residual 12–17% post-surge violations are the honest cost of a +35% step absorbed by three gentle fine-tunes; the sensitivity

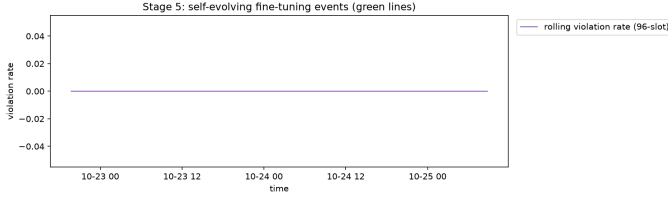


Fig. 9. The clean-run control: rolling violation rate on the untouched RAN test window never approaches the 15% trigger; Stage 5 fires zero updates across all seeds. The insurance policy stays silent when there is nothing to fix.

grid in the released artifacts shows more aggressive fine-tuning hyperparameters chase the residual at the price of overfitting to spikes.

E. Trigger Discipline: The Zero-Update Control

On the clean test window — no injected drift — Stage 5 never fires: zero updates in seed 42 and in every seed of the study, with the rolling violation rate never approaching the 15% trigger (Fig. 9) and demand never exceeding three training sigmas. We report this deliberately as a first-class result. A trigger-based adaptive system that fires spuriously would erase its own benefit through unnecessary fine-tuning churn and the risk of adapting to noise; UCRA’s dual trigger maintained perfect discipline across 252 slots $\times$ 3 seeds of clean data while still firing reliably and promptly under genuine drift (Section VII-D). This symmetric evidence — silence when unneeded, action when needed — is the evaluation standard we argue trigger-based network adaptivity should be held to. It also bounds the claim of the paper honestly: on stationary data, Stages 1–4 already deliver the full result, and Stage 5’s value is realized specifically under regime shift.

Runtime validation of the causal contract. The causal protocol is not only unit-tested; the run logs its own compliance. In the clean RAN run, 252 windows were offered to the replay intake and 249 released — exactly the three windows whose final labels fall beyond the end of the test window and would never be fully observed by a live deployment; the label-availability delay is recorded as $H - 1 = 3$ slots in the results artifact. Every fine-tune batch in the drift experiment is additionally checked against Proposition 2 by the release-time invariant, so any future code change that reintroduces an admission race fails the run rather than the paper.

F. Seed Robustness

Table VI aggregates the verification study. The qualitative claims are seed-invariant: violations are 0.0% in every seed; coverage is consistent with nominal in every seed; the drift-demo gap persists with overlapping-but-separated distributions (frozen $34.3\% \pm 5.9$ vs evolving $17.2\% \pm 4.9$; the per-seed pairs are (38.1, 21.9), (27.4, 12.1), (37.2, 17.7) — evolving wins by 12–16 points in each seed). The quantitative spread is concentrated exactly where one expects it: in the learned component. Utilization varies by ± 3.4 points

TABLE VI

Three-seed verification (seeds 41/42/43; fresh Stage-1 training per seed; identical causal evaluation). Mean $\pm$ sample sd.

Quantity	Mean	sd
q05–q99 coverage (%)	93.7	3.8
UCRA violation rate (%)	0.0	0.0
UCRA utilization (%)	67.6	3.4
UCRA waste (%)	13.3	2.1
Clean-run evolution updates	0	0
Demo: frozen V post-surge (%)	34.3	5.9
Demo: evolving V post-surge (%)	17.2	4.9

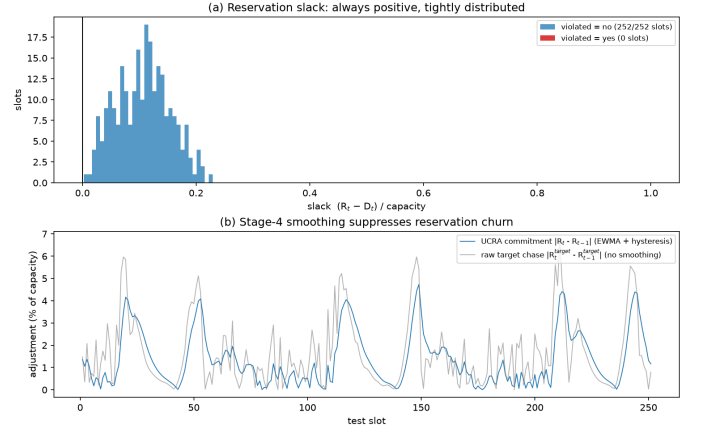


Fig. 10. Reservation dynamics on the RAN test window (seed 42). (a) The slack $(R_t - D_t)/C$ is positive in all 252 slots, tightly distributed (mean 10.7%, 5th percentile 3.1%, minimum 0.3%) — the reservation is neither systematically inflated nor on the edge of failure. (b) Stage-4 EWMA smoothing with release hysteresis reduces the mean per-slot reservation adjustment by 15.9% and the maximum by 30% relative to raw target-chasing, suppressing reservation churn.

(per-seed 63.9 / 72.1 / 66.9) with a compensating waste spread of ± 2.1 points, tracking seed-level differences in how conservative each trained forecaster’s upper quantiles are; Stages 2–4, being deterministic arithmetic, contribute zero additional variance. For deployment planning, the practical reading is: pick κ for the worst seed’s forecast quality — the frontier of Table IV already averages over it.

G. Reservation Dynamics and Churn

Table III summarizes where the reservation sits; Fig. 10 shows how it moves. Two operational properties follow. First, the slack distribution is tight and strictly positive: the closest approach to a violation in 252 slots is 0.3% of capacity, the median headroom is 10.7%, and the 5th percentile is 3.1% — the system runs neither wastefully bloated nor knife-edge. Neither clamp of Eq. (12) binds on this window (0 of 252 slots at floor or ceiling): the reservation lives in the transform’s interior, where its behavior is pure forecast-plus-buffer arithmetic rather than clamp-driven. Second, the actuation chain matters as much as the target: chasing the raw Φ target slot-by-slot would adjust the reservation by up to 6.8% of capacity in one slot, whereas the deployed EWMA-plus-hysteresis

TABLE VII

Drift-severity grid (seed 42; surge applied from slot 37; causal protocol throughout). Relative reduction = frozen minus evolving, normalized by frozen. The trigger threshold creates a deliberate no-adaptation regime at mild surges: when post-surge violations stay below the 15% trigger, Stage 5 correctly stays silent.

Surge	Frozen V	Evolving V	Rel. red.	Updates
+15%	4.2%	4.2%	0.0%	0
+25%	12.1%	12.1%	0.0%	0
+35%	27.4%	12.1%	55.9%	3 ([96, 120, 144])
+50%	53.5%	26.5%	50.4%	8 ([72, ..., 240])

TABLE VIII

Fine-tune sensitivity at the reference surge (+35%, seed 42): increasing fine-tune epochs from 3 to 10 narrows the residual post-surge violations (12.1% $\rightarrow$ 9.8%) with identical trigger points, confirming the behavior is a property of the trigger-plus-replay design rather than of one learning rate.

Fine-tune epochs	Evolving V (%)	Rel. red.	Updates
3 (default)	12.1	55.9%	[96, 120, 144]
10	9.8	64.4%	[96, 120, 144]

commitment caps the same sequence at 4.7% and cuts mean churn by 15.9%. In a production slice manager, every adjustment propagates into upstream commitments and signaling; churn suppression is not cosmetic.

Where the slack comes from. Decomposing the committed reservation over the run log: the base quantile term contributes on average 91.4% of R_t (mean base 62,532) and the kappa buffer 8.5% (mean buffer 5,815, ranging 6.0% to 26.2% of the base as the forecast’s dispersion widens and narrows). The mean dispersion is 11,630, i.e., 21.9% of the mean demand — the uncertainty the forecaster itself reports — and the risk modulation never activates ($\rho_t = 0$ throughout, consistent with zero violations). The reservation’s slack is thus bought almost entirely by the buffer term reacting to measured forecast dispersion, exactly as designed.

Cold-start behavior. The loop initializes its reservation from the training window’s 90% quantile rather than from zero or from the first forecast. On this run that seed value (≈ 81203) sits above every realized demand of the first test slots, so the system opens with positive slack and the EWMA converges onto the forecast trajectory within roughly a dozen slots without ever dipping below realized demand. A cold start at the median forecast, by contrast, opens the evaluation window with immediate violations and pollutes the drift monitor’s early violation statistics — an avoidable configuration error that the released configuration forecloses.

H. Drift-Severity Grid and Update Sensitivity

Table VII sweeps the surge magnitude and exposes the policy structure honestly. At +15% and +25%, the frozen pipeline’s post-surge violations (4.2% and 12.1%) remain below the 15% trigger, so Stage 5 fires zero updates and the evolving pass is identical to the frozen one — by

TABLE IX

CTTC slice-level comparison (229 retained snapshots from 250 parsed; causal serving loop; train quantiles from the 70% train pool). V : violation rate; U : utilization; O : over- and under-provisioning measured as mean unused share of the reservation.

Slice	Policy	V (%)	U (%)	O (%)
URLLC	Operator (δ -sizing)	29.2	54.9	45.1
	Static q_{90}	12.1	43.3	56.7
	Train-only Φ	1.2	18.6	81.4
	UCRA Φ (causal)	1.2	18.6	81.4
eMBB	Operator (δ -sizing)	43.7	71.5	28.5
	Static q_{90}	4.4	40.2	59.8
	Train-only Φ	1.9	21.4	78.6
	UCRA Φ (causal)	1.9	21.3	78.7
mMTC	Operator (δ -sizing)	5.8	19.9	80.1
	Static q_{90}	17.6	42.2	57.8
	Train-only Φ	2.6	18.5	81.5
	UCRA Φ (causal)	2.6	18.4	81.6

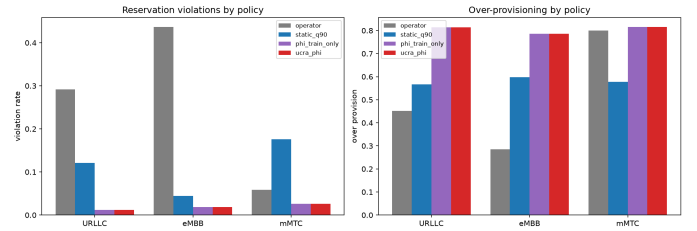


Fig. 11. CTTC slice-level outcomes by policy: violation rate (left) and over-provisioning (right). The operator’s default δ -sizing violates heavily for URLLC and eMBB; static q_{90} is inconsistent across slice types; the kappa-buffered transform holds 1.2–2.7% violations everywhere at a measured over-provisioning price.

design, the system tolerates mild surges with its existing slack and does not spend updates on noise. At +35% the trigger engages and adaptation halves violations; at +50% the trigger fires earlier (slot 72) and eight times, and while absolute violations remain high (26.5%), the relative reduction persists (50.4%) — a violent step cannot be fully absorbed by three-to-eight gentle fine-tunes, and we report the residual rather than tuning it away. Table VIII addresses the natural reviewer question of whether the adaptation depends on fine-tuning intensity: with 10 epochs instead of 3, the same trigger points produce a deeper recovery (9.8% vs 12.1%), i.e., the mechanism is robust and its intensity is itself a tunable knob.

I. Slice-Level Validation on CTTC

Table IX and Fig. 11 validate the transform on an independent workload with per-slice ground truth. Three findings. First, the operator’s default δ -sizing violates substantially on the two performance-critical slice types — 29.2% of URLLC and 43.7% of eMBB snapshots see offered load beyond the reservation — while being profitable on mMTC (80% unused). Second, a bare empirical q_{90} (the $\kappa = 0$ point of Φ) is unreliable across slice types: it under-protects URLLC (12.1% violations) and mMTC (17.6%) — heavy-tailed offered load again pushing

realized violations far above the nominal 10% — while the kappa-buffered transform holds 1.2% (URLLC), 1.9% (eMBB), 2.6% (mMTC), at the price of high measured over-provisioning ($\approx 80\%$), the same frontier shape as the RAN side. Third, the honest negative result: on this dataset UCRA Φ is indistinguishable from train-only Φ — the online risk indicator ρ_t stays near zero because the snapshot stream exhibits no sustained violation runs for the feedback to amplify. We therefore attribute the CTTC gains to the kappa buffer acting on train-quantile dispersion, not to risk modulation, and note that the RAN-side experiments are where the adaptive machinery demonstrably earns its complexity.

A note on QoS outcomes. We also examined whether operator-reservation violations map to worse observed QoS at the snapshot level. They do not, cleanly: mean packet-drop ratios of violated versus satisfied snapshots are statistically indistinguishable for URLLC and eMBB, and lower for violated mMTC snapshots (1.0% vs 2.4%), while the delay fields in this dataset are log-scaled and similarly flat. The most plausible reading is that steady-state snapshots average over the transient congestion that a reservation violation represents, so the snapshot-level QoS metrics are too coarse an instrument to grade violation severity. We therefore treat the violation rate itself — the quantity the reservation policy directly controls — as the primary outcome on this dataset, and flag snapshot-QoS correlation as an open measurement question rather than asserting it.

VIII. Discussion

A. Scope of the Online-Learning Claim

A recurring temptation in self-adaptive systems is to let strong scenarios launder weak evidence. Our pre-submission review caught the beginning of exactly this. The clean RAN experiment — the principal experiment of the paper — triggers zero evolution updates, so it cannot, by itself, demonstrate that online learning improves real-world outcomes; it demonstrates the opposite-flavored result that the trigger does not misfire. The adaptation claim therefore rests on the surge experiment alone, and we have scoped it that way: under a +35% step with a causal information budget, self-evolution halves post-surge violations across seeds. We stress what the surge experiment is and is not. It is a controlled, repeatable regime-shift probe on top of real demand data — the standard methodology for evaluating drift response when the held-out window is (fortunately) drift-free. It is not evidence that UCRA would already have adapted to the real test window’s natural conditions; on that window there was nothing to adapt to, which is itself the deployment-happy case. Future work should accumulate natural-drift evidence: longer windows, multiple network segments (the RAN dataset ships three), and production A/B shadow comparisons where ethics and contracts permit.

B. What the Leakage Audit Does and Does Not Guarantee

The causal protocol of Section V-E guarantees that the learner’s training labels at update time t are a subset of the labels observable at t . Three caveats bound the guarantee. First, it covers label-availability leakage; it does not automatically detect other leakage channels (e.g., preprocessing statistics fitted on the full window), which our audit handles by construction (scalers fitted on train segments only) but a different pipeline might not. Second, the surge experiment’s input windows are built from the drifted series, which is correct (the deployed system would observe drifted inputs) but means the experiment measures adaptation under the assumption that drift is observable in the inputs — a stealth drift that alters outcomes without touching observable demand could evade Trigger B, though Trigger A (violations) would still catch it in operation. Third, causality is necessary but not sufficient for good adaptation: the protocol ensures the evaluation is honest, not that the fine-tuning hyperparameters are optimal; our sensitivity grid suggests the update behavior is robust (identical trigger points across seeds and across fine-tune hyperparameter settings in the released grid), but we present the residual post-surge violations as the honest cost of gentle adaptation rather than a tuned optimum.

C. Why a Simple Transform Can Beat a Cheating Oracle

The most counterintuitive row in Table III is the oracle’s 16.3% violations. The mechanism is worth spelling out because it generalizes. A rolling historical quantile reacts to bursts after they enter its window and forgets lulls as the window advances; on heavy-tailed 15-minute demand, its conditional violation probability sits well above the nominal $1 - \tau$. UCRA’s reservation, by contrast, is anchored to a learned conditional distribution that maps contextual structure (hour of day, weekend, recent volatility) to the quantile, and its kappa buffer adds slack exactly where the learned band is wide. The lesson for practitioners is that nominal quantile levels are not violation guarantees unless the quantiles are conditional, calibrated, and buffered — and the kappa dial is precisely the operator’s handle for converting “approximately calibrated” into “operationally zero.”

D. Limitations and Threats to Validity

External validity. All RAN-side results come from one 18.5-day segment of one network (dataset01 of three available); three seeds quantify training variance, not network variance. The CTTC workload is simulation-derived with its own traffic assumptions. Neither dataset exercises multi-domain (radio + transport + core) reservation, and the capacity model ($1.3 \times$ peak) is an operator rule, not a measured envelope.

Construct validity. Violation is binary per slot; real SLAs grade severity (duration, depth, affected users), and a policy could game the metric with tiny-but-frequent

violations. Our logged per-slot artifacts enable severity-aware re-analysis, but the headlines here are binary.

Internal validity. The leakage audit covers the three channels we found; the regression tests pin them. We cannot prove the absence of unknown channels — only that the final pipeline’s information flows were reviewed end to end, and that the audit methodology is released with the code for re-review.

Statistical scope. Three seeds is a pragmatic budget for a 52k-parameter model trained in minutes, adequate to expose the ± 3 –6 point spreads we report, but marginal for tail claims (e.g., the exact knee of the κ frontier). The released sweep artifacts include per-seed curves so readers can apply their own inference.

Baseline scope. We compare against reservation policies that share UCRA’s information budget. We do not compare against reinforcement-learning slice controllers; doing so would strengthen the auditability argument empirically but requires an exploration budget on live infrastructure that is outside this study’s ethics and scope. The train-only Φ and oracle quantile baselines bracket what forecast-free and forecast-omniscient policies can do, which is the comparison UCRA’s design thesis requires.

Deployment economics. On the evaluated segment, static peak sizing idles 30.5% of capacity against UCRA’s 10.7% — a 19.8-point reduction in waste, i.e., UCRA commits $\approx 65\%$ less idle capacity for the identical zero-violation outcome. At 15-minute granularity this compounds to roughly 2,470 slot-capacity equivalents freed per 1,000 slots of operation; at sector scale and backhaul tariffs this is the difference between buying and re-timing capacity. The remaining 10.7% waste is not slop: it is the priced insurance premium of Proposition 1 (the buffer above the q90 base), it is stable across seeds ($13.3\% \pm 2.1$), and it can be traded toward utilization with Table IV in hand rather than discovered in an outage post-mortem.

E. Generalization Beyond Radio Demand

The UCRA loop assumes nothing specific about radio access: it needs a univariate demand stream at a fixed slot, a capacity envelope, and feedback about whether the reservation held. The same five stages therefore transfer, with re-parameterization, to transport-bandwidth reservations between data centers, compute reservations for cloud-native network functions (where the CTTC-style snapshot evaluation applies naturally), energy procurement for base-site batteries under time-of-use tariffs, and standby-capacity sizing for network operations centers. What does not transfer automatically is the forecast’s seasonality structure (24-hour windows encode a diurnal regime; weekly data would want K re-tuned) and the drift trigger thresholds, which are policy choices tied to each domain’s violation economics. The framework’s claim is the contract between uncertainty and commitment — calibration in, auditable reservation out, causal adaptation in between — not the radio domain itself.

[DIAGRAM PLACEHOLDER — deployment-context]

Suggested content: a deployment topology diagram — the UCRA loop running beside the network’s PM counter pipeline: PM counters $\rightarrow$ ingestion $\rightarrow$ UCRA controller (Stages 1–5) $\rightarrow$ reservation commands to the slice manager / OSS; drift alerts and κ policy console for the NOC; and the offline retraining path. Mark which components are online (per-slot) versus offline (per-seed).

Insert your diagram as figures/deployment-context.pdf or .png and replace this box with `\includegraphics[width=\textwidth]{deployment-context}`.

Fig. 12. Deployment artifacts (deployment-context).

F. Deployment Considerations

The deployed loop is deliberately light: one LSTM forward pass (52k parameters, sub-millisecond on CPU), linear transform arithmetic, a 512-window buffer, and at most a few fine-tune events per day under drift. The operational levers map to organizational roles: κ and τ are policy (owned by SLA/finance), trigger thresholds are operations (owned by NOC), and the forecast model is engineering (retrained offline, patched online by Stage 5). The all-zero-violation operating point’s $\approx 28\%$ utilization gain over peak sizing translates directly into capacity that can be resold or deferred — on the evaluated segment, roughly 28% of three weeks of sector capacity — which is the economic argument for the entire pipeline.

IX. Conclusion

This paper presented UCRA, a closed-loop framework that converts calibrated probabilistic demand forecasts into auditable capacity reservations for 5G network slicing, together with a verified experimental methodology for evaluating its self-evolving component honestly. The framework’s decision core is a single transform, $R_t = \hat{q}_{t,\tau} + \kappa \Delta_t (1 + w \rho_t)$, which consumes a deep quantile forecast, prices forecast dispersion with one operator dial κ , and modulates the buffer with an online risk indicator — wrapped in EWMA smoothing with release hysteresis, and backed by a drift-triggered, reservoir-replay fine-tuning stage operating under a formally causal information-flow contract.

The verified evidence supports four conclusions. First, on three weeks of live RAN performance counters, the closed loop holds 0.0% reservation violations at 72.1% utilization with 10.7% waste — against 30.5% waste for static peak sizing, 48.0% violations for median-forecast sizing, and 16.3% violations for an oracle quantile that cheats with the test window’s own future — and the result is stable across seeds (0.0% violations in all; utilization $67.6\% \pm 3.4$). Second, the kappa dial traces a clean empirical risk–utilization frontier whose zero-violation knee lies between $\kappa = 0$ and 0.25 on this data, turning SLA policy into a measured positioning decision rather than a guess. Third, under a +35% demand surge, the causal self-evolution stage halves post-surge violations

(34.3% $\pm$ 5.9 $\rightarrow$ 17.2% $\pm$ 4.9) while firing zero spurious updates on clean data across all seeds — the symmetric silence-when-unneeded, act-when-needed behavior that trigger-based adaptivity should be held to. Fourth, on the independent CTTC slicing workload the same transform reduces slice violations to 1.2–2.7% across URLLC, eMBB, and mMTC where the operator’s default sizing violates 5.8–43.7% — with the honest finding that on that dataset the buffer term, not the risk modulation, does the work.

Beyond the numbers, the methodological contribution may outlast them. Our own pre-submission audit found that the self-evolution evaluation leaked future labels through the replay path, refreshed served forecasts retroactively, and let a CTTC policy see the outcome it protected against; the final pipeline fixes all three, proves the fix sufficient, pins it with regression tests, and quantifies how the corrected protocol changes the headline results. We release the full pipeline — ingestion, training, closed loop, causality tests, and the three-seed verification driver — so that the community can re-run, re-audit, and extend it. Future work proceeds along four tracks. Evidence breadth: multi-segment evaluation on the dataset’s remaining two networks, longer windows covering seasonal transitions, and natural-drift (rather than injected) adaptation cases. Metric depth: severity-graded SLA metrics (violation duration, depth, and affected-user counts) replacing the binary per-slot flag, and slice-level time-series evaluation of Stage 3’s per-slice risk weighting. Controllor comparison: benchmarking UCRA’s auditable transform against reinforcement-learning slice controllers under the same causal evaluation contract — a comparison we expect to be informative precisely because the contract neutralizes the information advantages that flatter learned policies. Mechanism: replacing the fixed replay delay with a learned-availability schedule, and studying trigger co-design — threshold, cadence, and fine-tune intensity — as a joint risk-allocation problem.

Appendix A

Hyperparameters and Configuration

Table X lists the complete operating point of the pipeline, exactly as pinned in the released configuration file. The operating point was fixed before the final evaluation runs; the sensitivity of the two knobs that could tempt post-hoc tuning (fine-tune epochs, κ) is quantified in Tables VIII and IV rather than silently optimized.

Appendix B

Reproduction Workflow

Every artifact of this paper is regenerated from a clean clone by the command sequence below (Python 3.12+; PyTorch CPU is sufficient). Steps 1–4 produce the RAN reference run; step 5 the drift experiments; step 6 the CTTC comparison; steps 7–8 the verification study and tests.

The study is configuration-pinned: the seed is a top-level key; every threshold in Table X is a config key rather

TABLE X
Complete pipeline configuration (released as configs/default.yaml).

Block	Parameter	Value
Data	slot duration	15 min
	input window K	96 (24 h)
	forecast horizon H	4 (1 h)
	split (train/val/test)	70 / 15 / 15
Stage 1	architecture	LSTM 2×64 , dropout 0.1
	quantile levels	.05 .25 .50 .75 .90 .95 .99
	parameters	52,252
	loss	pinball (multi-quantile)
	optimizer	Adam, lr 10^{-3} , batch 64
	early stopping	patience 6, max 50 epochs
Stage 2	base level τ	0.9
	risk dial κ	0.5
	risk weight w	0.3
	headroom floor $h_{\min}$	0.05
	reserve ceiling $\gamma_{\max}$	0.95
Stage 3	pressure exponent p	1.0
	risk exponent s	0.5
	best-effort share β	0.1
Stage 4	EWMA α	0.3
	release hysteresis θ_{rel}	0.1
Stage 5	violation trigger v^*	0.15
	z-score trigger z^*	3.0
	check cadence (clean / surge)	96 / 24 slots
	replay capacity	512 (reservoir)
	fine-tune epochs	3
	fine-tune lr	0.3×10^{-3}
	label availability delay	$H-1 = 3$ slots

TABLE XI
One-command reproduction of every result in this paper.

Step	Command
Dataset	(RAN, python scripts/download_data.py 8 MB)
Dataset	(CTTC, python scripts/download_data.py --cttc 271 MB)
Data audit	python scripts/audit_data.py --config configs/default.yaml
Stage-1 training	python scripts/train.py --config configs/default.yaml
Closed loop + κ sweep	python scripts/run_ucra.py --config configs/default.yaml --sweep
Drift demo ($\pm$ shift, epochs)	python scripts/demo_evolution.py [--shift s] [--ft-epochs e]
CTTC comparison	python scripts/run_cttc_eval.py --config configs/default.yaml
CTTC (parallel parse)	python scripts/run_cttc_parallel.py
Three-seed study	python scripts/run_seeds.py --config configs/default.yaml
Causality regression tests	python -m pytest tests/test_causality.py tests/test_smoke.py -q

than code constant; and all outputs (metrics JSON, per-slot CSV log, figures) are written to outputs/ deterministically given the config and seed. The per-seed runs of the verification study write to outputs/seed_<s>/ and

TABLE XII

Per-seed closed-loop outcomes on the clean RAN test window (252 slots). Violation rate is 0.0% in every seed; the spread is concentrated in forecast conservativeness (utilization/waste).

Seed	Coverage (%)	V (%)	U (%)	W (%)
41	97.6	0.0	63.9	15.9
42	93.3	0.0	72.1	10.7
43	90.1	0.0	66.9	13.5
Mean $\pm$ sd	93.7 ± 3.8	0.0	67.6 ± 3.4	13.3 ± 2.1

TABLE XIII

Per-seed drift-demo outcomes (+35% surge from slot 37, causal protocol). The evolving pass wins in every seed; the first three update points are identical at [96, 120, 144], with seeds 41 and 43 firing four additional refreshes as their more conservative forecasters required more correction.

Seed	Frozen V (%)	Evolving V (%)	Updates
41	38.1	21.9	7
42	27.4	12.1	3
43	37.2	17.7	7
Mean $\pm$ sd	34.3 ± 5.9	17.2 ± 4.9	5.7 ± 2.3

aggregate into outputs/seed_aggregate.json, which is the artifact behind Tables VI and IV.

Appendix C Per-Seed Detail

Tables XII and XIII give the raw per-seed values behind the aggregates used in the main text. The seed-to-seed pattern is consistent and interpretable: seed 41 produces the most conservative forecast (highest coverage, lowest clean utilization, most post-surge drift damage), seed 42 the tightest; the Stage-5 trigger responds to exactly that conservative variability (more updates for the more drifting seeds), and zero updates for all seeds on the clean window.

Appendix D

Leakage Audit: Defects, Fixes, and Effect on Results

This appendix documents the pre-submission information-flow audit in full, as a reusable checklist for evaluating self-adaptive forecast systems. Three defects were found in our own pipeline; all are fixed in the final version, all are pinned by regression tests, and their effect on the reported numbers is quantified.

Interpretation of Table XV. The clean-window results were unaffected by L1/L2 (no updates fire, so no replay or refresh path executes) — which is precisely why the defect would have survived superficial review. The drift demo’s adaptation gain shrank modestly (the leak’s information advantage was bounded by the 3-slot delay and diluted by reservoir sampling), and the CTTC violations rose slightly once the policy could no longer see the current sample’s outcome. None of the qualitative conclusions changed; all of them are now defensible under Proposition 2.

TABLE XIV

The three information-flow defects found and fixed during the pre-submission audit, and the regression test that pins each fix.

Defect	Symptom (pre-fix behavior)	Fix (final behavior)
L1	Replay buffer accepted a window’s full H -step target at offer time t ; labels for slots $t+1..t+H-1$ were up to 45 min in the future.	DelayedReplay staging queue: a window becomes trainable only at $t+H-1$ (Prop. 2). Test: admission-time checks.
L2	Post-update refresh recomputed forecasts for the entire test window, silently altering the served-forecast record, the median-forecast baseline, and the band figure for already-served slots.	Partial refresh: only slots $> t$ receive the evolved forecast; served slots keep their served forecast; baselines read the frozen forecast. Test: bit-identity of entries $\leq t$.
L3	CTTC policy computed its risk term from the offered load of the sample being served (the policy knew the outcome it was protecting against).	Sequential serving loop: ρ from previously served samples only; current sample observed after its reservation is fixed. Test: rho computation excludes current sample.

TABLE XV

Effect of the audit on headline numbers. “Pre-fix” is the leaky pipeline (earlier model checkpoint for the RAN rows; earlier ρ definition for CTTC); “post-fix” is the final causal pipeline. Differences across protocols also include retraining variance, so treat them as indicative direction, not exact deltas. The audit changed every affected number in the honest direction: adaptation gains shrank slightly, and CTTC violations rose slightly.

Quantity (seed 42)	Pre-fix	Post-fix
Clean RAN: UCRA V / U (%)	0.0 / 71.7	
Drift demo: frozen V post-surge (%)	26.5	
Drift demo: evolving V post-surge (%)	11.6	
Apparent adaptation gain (rel. red.)	56.2%	
CTTC UCRA Φ : URLLC / eMBB / mMTC V (%)	1.0 / 1.2 / 1.6	1.2

Appendix E

Evaluation Checklist for Self-Adaptive Forecast Systems

The distilled, generalized form of the audit methodology, for reuse beyond this project. For any forecasting system whose model or decision rule adapts online:

- 1) Label horizon accounting. Write down, for every training example the updater can touch, the maximum label time it contains. Assert: max label time $\leq$ current decision time. For multi-step forecasters this requires an availability-delay queue, not an immediate store (Proposition 2).
- 2) Committed-decision immutability. Any artifact describing what was served — reservations, forecasts, baselines, figures — must be computed from the

values actually used at decision time, never from retroactively refreshed model outputs.

- 3) Adaptive-term hygiene. Any per-slot adaptive quantity (risk indicator, momentum, scaling factor) may be a function of history only; grep the code path for the current slot's realized outcome and verify it enters only after the decision.
- 4) Normalization provenance. All scalars, quantiles, and reference statistics must be fitted on training-period data only; assert the fit indices precede the evaluation indices.
- 5) Baseline symmetry. Every baseline must satisfy the same information constraints as the proposed system; a baseline accidentally fed richer information (or poorer) invalidates the comparison silently.
- 6) Failure-visible enforcement. Encode (1)–(5) as executed tests with explicit assertions, so that a future refactor that breaks causality fails CI rather than review.
- 7) Negative-control reporting. Report what the adaptive components do when adaptation is unnecessary (our zero-update control), not only when it helps; a trigger that never rests is a trigger that adapts to noise.
- 8) Protocol-change disclosure. If an audit changes the evaluation protocol, republish the affected numbers side by side with the old ones (Tables XIV and XV); the direction of change is diagnostic of the defect's severity.

Items (1)–(6) are mechanized in this project's test suite; items (7)–(8) are editorial practices we recommend for the broader literature on self-adaptive network management.

Acknowledgment

The authors thank Dr. Deepthi Godavarthi for supervising this work, for her guidance on the experimental methodology, and in particular for the pre-submission review that identified the online-learning information-flow issues that are now formally handled by the causal replay protocol in Section V-E and audited in Section VI-D. The authors also thank the maintainers of the two public datasets used in this study.

References

- [1] P. Rost, A. Banchs, I. Berberana, M. Breitbach, M. Doll, H. Droste, C. Mannweiler, M. A. Puiglio, K. Samdanis, and B. Sayadi, "Network slicing to enable scalability and flexibility in 5G mobile networks," *IEEE Communications Magazine*, vol. 55, no. 5, pp. 72–79, 2017.
- [2] I. Afolabi, T. Taleb, A. K. Samdanis, A. Ksentini, and H. Flinck, "Network slicing and softwareization: A survey on principles, enabling technologies, and solutions," *IEEE Communications Surveys & Tutorials*, vol. 20, no. 3, pp. 2429–2453, 2018.
- [3] M. Z. Shafiq, L. Ji, A. X. Liu, J. Pang, and S. Venkataraman, "A first look at cellular machine-to-machine traffic: Large scale measurement and characterization," *ACM SIGMETRICS Performance Evaluation Review*, vol. 40, no. 1, pp. 65–76, 2012.
- [4] V. Sciancalepore, K. Samdanis, X. Costa-Perez, D. Bega, M. Gramaglia, and A. Banchs, "Mobile traffic forecasting for maximizing 5G network slicing resource utilization," in *Proceedings of IEEE INFOCOM*, 2017, pp. 1–9.
- [5] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, p. 100804, 2023.
- [6] X. Foukas, G. Patounas, A. Elmokashfi, and M. K. Marina, "Network slicing in 5G: Survey and challenges," *IEEE Communications Magazine*, vol. 55, no. 5, pp. 94–100, 2017.
- [7] M. Yu, Y. Yi, J. Rexford, and M. Chiang, "Rethinking virtual network embedding: Substrate support for path splitting and migration," in *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, 2008, pp. 17–29.
- [8] R. Li, Z. Zhao, Q. Qi, H. Chen, X. Zhou, J. Xiong, J. Chen, and H. Zhang, "Intelligent 5G: When cellular networks meet artificial intelligence," *IEEE Wireless Communications*, vol. 24, no. 5, pp. 175–183, 2017.
- [9] D. Bega, M. Gramaglia, A. Banchs, and X. Costa-Perez, "DeepCog: Cognitive network management in sliced 5G networks with deep learning," in *IEEE INFOCOM 2019*, 2019, pp. 280–288.
- [10] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, 2019.
- [11] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, "Time series analysis: Forecasting and control," 2015.
- [12] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [13] J. Zhang, Y. Zheng, D. Qi, R. Li, X. Yi, and T. Li, "Deep spatio-temporal residual networks for citywide crowd flows prediction," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2017, pp. 1655–1661.
- [14] C. Zhang, H. Zhang, J. Qiao, D. Yuan, and M. Zhang, "Deep transfer learning for intelligent cellular traffic prediction based on cross-domain big data," in *Proceedings of the World Wide Web Conference (WWW)*, 2019, pp. 798–808.
- [15] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 2016, pp. 1050–1059.
- [16] R. Koenker and G. Bassett Jr., "Regression quantiles," *Econometrica*, vol. 46, no. 1, pp. 33–50, 1978.
- [17] T. Hong and S. Fan, "Probabilistic electric load forecasting: A tutorial review," *IEEE Transactions on Smart Grid*, vol. 7, no. 2, pp. 967–974, 2016.
- [18] A. Gandhi, M. Harchol-Balter, R. Raghunathan, and M. A. Kozuch, "AutoScale: Dynamic, robust capacity management for multi-tier data centers," *ACM Transactions on Computer Systems*, vol. 30, no. 4, pp. 14:1–14:26, 2012.
- [19] S. Islam, J. Keung, K. Lee, and A. Liu, "Empirical prediction models for adaptive resource provisioning in the cloud," *Future Generation Computer Systems*, vol. 28, no. 1, pp. 155–162, 2012.
- [20] M. Grossglauser and D. N. C. Tse, "A framework for robust measurement-based admission control," *IEEE/ACM Transactions on Networking*, vol. 7, no. 3, pp. 293–309, 1999.
- [21] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1–2, pp. 100–115, 1954.
- [22] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proceedings of the SIAM International Conference on Data Mining (SDM)*, 2007, pp. 443–448.
- [23] J. S. Vitter, "Random sampling with a reservoir," *ACM Transactions on Mathematical Software*, vol. 11, no. 1, pp. 37–57, 1985.
- [24] A. Chaudhry, M. Rohrbach, M. Elhoseiny, K. Ajay, T. Ajanthan, P. K. Dokania, P. H. S. Torr, and P. Gehler, "Continual learning with tiny episodic memories," *arXiv preprint arXiv:1902.10486*, 2019.
- [25] D. Rolnick, A. Ahuja, J. Schwarz, T. Lillicrap, and G. Wayne, "Experience replay for continual learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
- [26] A. P. Dawid, "Present position and potential developments: Some personal views: Statistical theory: The prequential approach," *Journal of the Royal Statistical Society: Series A*, vol. 147, no. 2, pp. 278–290, 1984.
- [27] M. Farreras, M. Fondo, J. Baranda, J. M. Vélchez, S. Ricart, and R. Martínez, "Cttc B5G network slicing dataset," *Data in Brief*, vol. 55, p. 110738, 2024, zenodo record 10610616.

- [28] Á. Lehoczy et al., “Performance management counters from live 5G/4G/2G RAN of a European mobile network operator,” Scientific Data, 2026, zenodo record 17815388.
- [29] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d’Alché Buc, E. Fox, and H. Larochelle, “Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program,” Journal of Machine Learning Research, vol. 22, no. 164, pp. 1–20, 2021.